



PDF Download
3744904.pdf
17 March 2026
Total Citations: 0
Total Downloads:
1020

Latest updates: <https://dl.acm.org/doi/10.1145/3744904>

RESEARCH-ARTICLE

CGCGraph: Efficient CPU-GPU Co-execution for Concurrent Dynamic Graph Processing

YIMING SUN, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

JIE ZHANG, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

HUAWEI CAO, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

YUAN ZHANG, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

XUEJUN AN, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

JUNYING HUANG, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

View all

Open Access Support provided by:

Institute of Computing Technology Chinese Academy of Sciences

Published: 17 September 2025

Online AM: 16 June 2025

Accepted: 27 May 2025

Revised: 29 November 2024

Received: 31 July 2024

Citation in BibTeX format

CGCGraph: Efficient CPU-GPU Co-execution for Concurrent Dynamic Graph Processing

YIMING SUN, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China, Zhongguancun Laboratory, Beijing, China, and University of the Chinese Academy of Sciences School of Computer Science and Technology, Beijing, China

JIE ZHANG, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China

HUAWEI CAO, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and Zhongguancun Laboratory, Beijing, China

YUAN ZHANG, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China

XUEJUN AN, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and Zhongguancun Laboratory, Beijing, China

JUNYING HUANG, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and Shanghai Innovation Center for Processor Technologies, Shanghai, China

XIAOCHUN YE, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China

With the continuous growth of user scale and application data, the demand for large-scale concurrent graph processing is increasing. Typically, large-scale concurrent graph processing jobs need to process corresponding snapshots of dynamically changing graph data to obtain information at different time points. To enhance the throughput of such applications, current solutions concurrently process multiple graph snapshots on the GPU. However, when dealing with rapidly changing graph data, transferring multiple snapshots of concurrent jobs to the GPU results in high data transfer overhead between CPU and GPU. Additionally, the execution mode of existing work suffers from underutilization of GPU computational resources.

In this work, we introduce CGCGraph, which can be integrated into existing GPU graph processing systems like Subway, to enable efficient concurrent graph snapshot processing jobs and enhance overall system resource utilization. The key idea is to offload unshared graph data of multiple concurrent snapshots to the CPU, reducing CPU-GPU transfer overhead. By implementing CPU-GPU co-execution, there is potential for

Yiming Sun and Jie Zhang contributed equally to this research.

This work was supported by the National Key Research and Development Program of China (2023YFB4502305), the Beijing Natural Science Foundation (4232036), and the CAS Project for Youth Innovation Promotion Association.

Authors' Contact Information: Yiming Sun, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China, Zhongguancun Laboratory, Beijing, China, and University of the Chinese Academy of Sciences School of Computer Science and Technology, Beijing, China; e-mail: sunyiming20g@ict.ac.cn; Jie Zhang, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China; e-mail: zhangjie2023@ict.ac.cn; Huawei Cao (corresponding author), Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and Zhongguancun Laboratory, Beijing, China; e-mail: caohuawei@ict.ac.cn; Yuan Zhang, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China; e-mail: zhangyuan-ams@ict.ac.cn; Xuejun An, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and Zhongguancun Laboratory, Beijing, China; e-mail: axj@ict.ac.cn; Junying Huang, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and Shanghai Innovation Center for Processor Technologies, Shanghai, China; e-mail: huangjunying@ict.ac.cn; Xiaochun Ye, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China; e-mail: yexiaochun@ict.ac.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 1544-3973/2025/09-ART91

<https://doi.org/10.1145/3744904>

enhanced utilization of GPU computing resources. Specifically, CGCGraph leverages kernel fusion to process shared graph data concurrently on the GPU, while executing all snapshots in parallel on the CPU, with each snapshot assigned a dedicated thread. This approach enables efficient concurrent processing within a novel CPU-GPU co-execution model, incorporating three optimization strategies targeting storage, computation, and synchronization. We integrate CGCGraph with Subway, an existing system designed for out-of-GPU-memory static graph processing. Experimental results show that the integration of CGCGraph with current GPU-based systems obtains performance improvements ranging from 1.7 to 4.5 times.

CCS Concepts: • **Computing methodologies** → **Parallel computing methodologies**; • **Information systems** → *Information systems applications*;

Additional Key Words and Phrases: CPU-GPU co-execution, concurrent graph processing, dynamic graph snapshot processing, high throughput

ACM Reference Format:

Yiming Sun, Jie Zhang, Huawei Cao, Yuan Zhang, Xuejun An, Junying Huang, and Xiaochun Ye. 2025. CGC-Graph: Efficient CPU-GPU Co-execution for Concurrent Dynamic Graph Processing. *ACM Trans. Arch. Code Optim.* 22, 3, Article 91 (September 2025), 26 pages. <https://doi.org/10.1145/3744904>

1 Introduction

Nowadays, graph processing algorithms, represented by SSSP and BFS [44, 45], have been increasingly researched because graphs can naturally model the relationships between real-world entities, such as road navigation, social media, online shopping, and financial risk control [7, 13, 14, 35]. To enhance their performance, numerous GPU-based graph processing systems have been developed, including Gunrock [39], SEP-Graph [36], LargeGraph [47], and Subway [29]. Since real-world graphs evolve over time, dynamic graphs are widely used to better represent realistic scenes [17, 24, 27]. However, the GPU-based graph processing systems mentioned above are primarily designed to enhance the performance of individual graph processing jobs, but demonstrate unsatisfactory efficiency in handling dynamically evolving graph data. To handle rapidly evolving graphs, there are usually multiple jobs concurrently processing different graph snapshots with different timestamps to obtain time-specific information, typically resulting in severe performance degradation as concurrency increases. Taking the SSSP algorithm as an example, the average overlap of active vertices in each iteration across consecutive snapshots over different datasets ranges from 42.4% to 66.0% according to reference [33]. In this scenario, multiple graph processing jobs accessing distinct snapshots may repeatedly load identical graph data into the GPU through different access paths, leading to redundant CPU-GPU transfers that significantly impact execution time. To address this problem, EGraph [46] proposes the **Loading-Processing-Switching (LPS)** execution model to enable sharing of graph data among concurrent jobs. First, EGraph extracts shared partitions from consecutive snapshots and transfers them to the GPU for concurrent processing. Next, unshared partitions are transferred to the GPU sequentially and processed serially. This approach minimizes transfer overhead by eliminating the need to move entire graph snapshots to the GPU.

However, in certain fields, such as financial risk control, graph data may change even more dramatically. According to Alibaba, on November 11, 2020, peak transactions reached 583,000 per second, causing significant financial graph fluctuations. In this similar scenario where the graph data changes rapidly, the LPS execution mode exhibits certain limitations, primarily in the following two aspects: (1) **High unshared data transfer overhead**: The sequential transfer of unshared partitions of the snapshots to the GPU incurs considerable CPU-GPU transfer costs, creating challenges in dynamic graphs with frequent changes. (2) **Low resource utilization**: Under-utilization

of GPU resources arises when different graph processing jobs consecutively execute shared partitions on the GPU. And the problem is even worse when graphs change rapidly. Additionally, the CPU remains relatively idle during graph processing compared to the heavily loaded GPU, highlighting an opportunity to improve efficiency through CPU-GPU co-execution.

To optimize CPU resource utilization, minimize CPU-GPU transfer overhead, and enhance GPU efficiency, we propose a software runtime system **CGCGraph**. The key idea of CGCGraph is to offload a portion of the computational workload from the GPU to the CPU, thus enabling efficient execution of concurrent jobs. CGCGraph divides a batch of consecutive graph snapshots into a shared graph and several unshared graphs, transferring the shared graph to the GPU while keeping unshared graphs on the CPU. It dynamically compacts the shared graph based on active edges and partitions it based on GPU capacity. Partitions are transferred from the CPU to the GPU in the common order. For one partition transferred to the GPU, jobs that need to process active edges in this partition are triggered, enabling multiple jobs processing the partition concurrently. When processing is completed, the next shared partition is continuously transferred to the GPU. Meanwhile, multiple CPU threads process unshared graphs in parallel. While the GPU concurrently handles shared graph partitions, the CPU runs separate threads for unshared graphs across jobs. Data synchronization occurs once both CPU and GPU computations are complete. In this way, the CPU and GPU can perform computations simultaneously, eliminating the need of transferring unshared graphs to the GPU for execution and reducing the transfer overhead between the CPU and GPU. Offloading serial computations on the GPU to the CPU enables parallel processing via multi-threading, improving overall resource utilization. The contributions of this article concerning CPU-GPU co-execution in dynamic graph processing can be summarized as follows:

- We reveal redundant data transfer overhead and low GPU resource utilization during concurrent processing of multiple dynamic graph snapshots on GPUs. Our analysis attributes these inefficiencies to the handling of unshared graphs, including transfers and computations.
- We propose an efficient CPU-GPU co-execution model for concurrent dynamic graph processing. Using this model, CGCGraph reduces the CPU-GPU data transfer overhead and improves the CPU and GPU resource utilization by transferring the shared graph to the GPU side and keeping unshared graphs on the CPU side for execution.
- We apply this CPU-GPU co-execution model in CGCGraph and implement three optimizing techniques to achieve more efficient CPU-GPU co-execution graph processing. Through optimizing techniques, we make up for the shortcomings of CGCGraph in storage, computation, and synchronization.
- Experiments are conducted by integrating CGCGraph into Subway, the state-of-the-art out-of-GPU memory processing system. Experimental results show that CGCGraph improves the overall performance by 1.7 to 4.5 times when executing dynamic graph snapshot processing jobs.

2 Background and Motivation

2.1 Discrete Evolving Graphs

The dynamic graph can be represented as a dynamic network that changes over time, and it can be classified into *temporal network* and *evolving network* depending on the duration of the links in the network [31]. Temporal networks have short event duration, while evolving networks are relatively stable, in which events last long enough so that network structures can be built. Dynamic networks can also be classified into *discrete network* and *continuous network* in terms of temporal granularity [31]. Discrete networks can be represented through a series of snapshots taken at

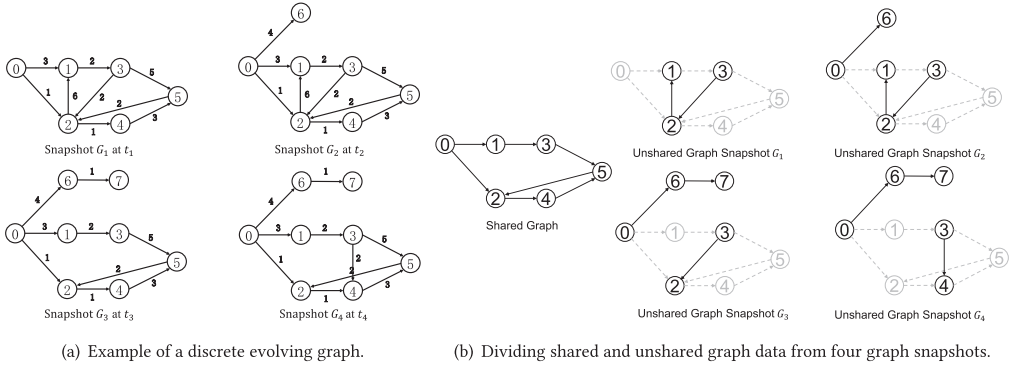


Fig. 1. Graph structure data of dynamic graph snapshot $G = \{G_1, G_2, G_3, G_4\}$.

distinct time intervals. Continuous networks carry more information, but their representation is more complex. In this article, we focus on the discrete evolving network.

We use $G = \{G_1, G_2, \dots, G_n\}$ to describe a set of discrete evolving graphs including n snapshots. The structural changes between two neighboring snapshots, G_{i-1} and G_i are typically represented by the addition or deletion of a set of edges or vertices (the update operation can be thought of as a deletion followed by an addition). Figure 1(a) shows a discrete evolving graph consisting of snapshots at four timestamps t_1, t_2, t_3 , and t_4 .

2.2 Vertex-centric Graph Processing

In this work, we focus on vertex-centric graph algorithms. Following the idea of “thinking like a vertex” [22], many graph processing systems employ the vertex-centric computing model [12, 18, 20, 30]. As shown in Algorithm 1, the vertex-centric computing model employs an “iterative-convergent” process. The vertices which need to be processed in the current iteration are referred to as active vertices, and all outgoing edges of active vertices are donated as active edges. In particular, the given graph algorithm is processed in an iterative manner, terminating when there are no more active vertices to be processed or when the procedure reaches the limited number of iterations. In each iteration, the Filter-Expand operation is applied to process all vertices in the graph. The filter step involves selecting active vertices from the set for expand (line 2), while inactive vertices are filtered out. The expand step processes the active vertices and generates new ones for the next iteration. It maintains algorithm-specific logic to ensure consistent processing of all active vertices with the same logic. Algorithm 1 illustrates the processing of expanding in the SSSP algorithm, which calculates the shortest distance from a given source vertex to all other vertices.

2.3 Concurrent Graph Processing

There are several concurrent graph processing systems to support the efficient operation of concurrent jobs, such as Seraph [40], CGraph [48], GraphM [50], Krill [8], Glign [43], and EGraph [46]. Seraph implements graph-level sharing, allowing concurrent queries to share common graph structure data. CGraph and GraphM package multiple queries into a single job and implement concurrent graph processing on the top of GridGraph [51]. They employ partition-level sharing to maximize the exploitation of temporal and spatial locality in accessing shared graphs. Both Krill and Glign extend the Ligra [30] framework. Krill points out the shortcomings of the coarse-grained partition-level sharing approach in CGraph and GraphM, and proposes a finer-grained vertex-level sharing model. Glign is optimized for monotonic algorithms only and further improves performance by merging frontiers across jobs and aligning heavy iterations across

ALGORITHM 1: Vertex-centric computing model (SSSP)

```

1: while  $\text{len}(\text{active vertex set}) > 0$  or  $\text{!convergent}$  do
2:   for  $v_s$  in active vertex set do
3:      $\text{EXPAND}(v_s, \text{next active vertex set})$ 
4:   end for
5:   active vertex set = next active vertex set
6:   next active vertex set = []
7: end while
8: function  $\text{EXPAND}(v_s, \text{next active vertex set})$ 
9:   for  $v_d$  in neighbor of  $v_s$  do
10:    distance =  $v_s.\text{value} + v_d.\text{weight}$ 
11:    if distance <  $v_d.\text{value}$  then
12:       $v_d.\text{value} = \text{distance}$ 
13:      next active vertex set =  $\text{append}(\text{next active vertex set}, v_d)$ 
14:    end if
15:  end for
16: end function

```

concurrent jobs. EGraph is a concurrent dynamic graph processing system on GPUs based on Subway [29]. EGraph enables concurrent processing of graph snapshot shared partitions through the LPS execution model, and dynamically schedules jobs across **streaming multiprocessors (SMs)** on GPUs for workload balancing.

In summary, the key to concurrent graph processing is to identify graph data that can be shared. Once the shared data is found, graph access can be shared across multiple jobs via the storage hierarchy.

Generally, a static graph is represented as $G = \{V, E, W, S\}$, where V is the vertex set, E is the edge set, W is the edge weights, and S is the vertex states. Each concurrent job has its own state data and iteratively updates the corresponding S until the computation converges. Since concurrent jobs are running on the same underlying graph, the shared data is the graph structure data $G_{\text{shared}} = \{V, E, W\}$, and the unshared data is the job-specific vertex state data $G_{\text{unshared}} = \{S\}$. Intuitively, each job has a job-specific vertex state array.

However, dynamic graph snapshots introduce additional complexity because the underlying graph structure evolves. While most graph data remains unchanged between consecutive snapshots, only a small portion is updated. Hence, the shared data of a dynamic graph snapshot is no longer the entire graph structure data. We use G_{shared} to represent the shared data, specifically referring to the graph structure data that is consistent across n dynamic graph snapshots. Graph structure data that varies across jobs and the job-specific vertex state data are unshared among multiple dynamic graph processing jobs. Because the optimizing strategies of these two are different, we use unshared data G_{unshared} to represent graph structure data that varies across jobs. And the structure of job-specific vertex state data will be optimized in Section 4. Naturally, n dynamic graph processing jobs have a shared data G_{shared} and n unshared data G_{unshared} , where n is the number of dynamic graph snapshots. Taking the four graph snapshots in Figure 1(a) as an example, Figure 1(b) shows one shared data and four unshared data.

2.4 Motivation

As described below, there are two primary limitations in existing concurrent graph processing systems, motivating our research on CGCGraph.

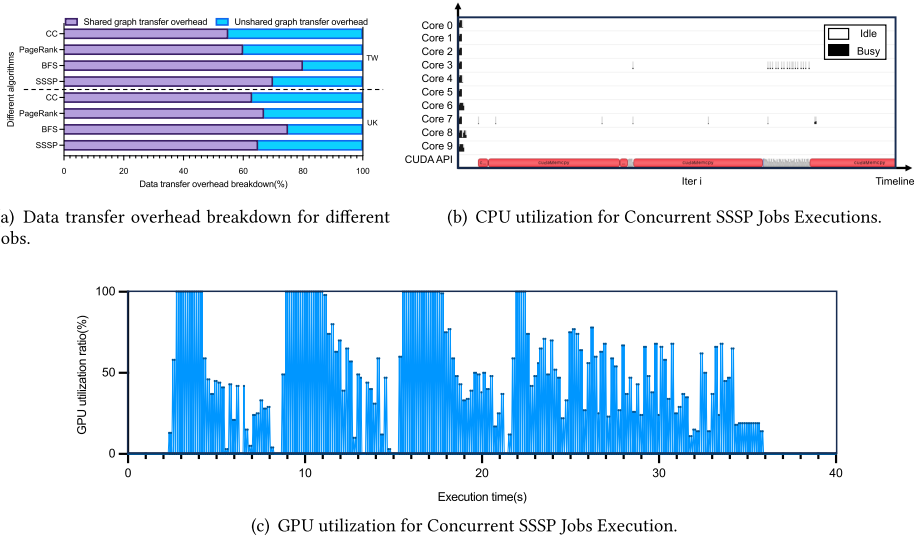


Fig. 2. Performance analysis of concurrent dynamic graph processing jobs executed on GPUs.

High unshared data transfer overhead. In current solution, both shared and unshared graph data must be transferred to the GPU for computation [46]. During computation, the shared graph is transferred to the GPU only once and processed by n jobs concurrently. But the unshared graphs of the n snapshots need to be sequentially transferred to the GPU, and the corresponding job will be triggered in sequence to process the unshared graphs. Although the transfer of unshared graphs consumes CPU-GPU bandwidth, it requires comparatively fewer computation resources due to the small unshared graph of a single job.

To illustrate the limitation of this execution scheme, experiments are conducted on the twitter-2010 and uk-2007 datasets, as depicted in Figure 2(a). Dynamic graph snapshot generation follows the methodology in Section 5. The batch size is set to 8, meaning 8 concurrent jobs, with a snapshot change rate of 1% (including edge insertions and deletions). As shown in Figure 2(a), unshared data transfer accounts for 20% to 45% of the total transfer overhead, significantly consuming CPU-GPU bandwidth. Additionally, even when considering overlapping computation and data transfer, the impact on latency remains minimal, as computation time is significantly shorter than communication time, ensuring that conclusions remain valid.

Notably, the partitioning method of the current solution will further increase the proportion of unshared data. EGraph's strategy involves an initial fine-grained partitioning, identifying partitions affected by graph updates as unshared, and the rest as shared. However, the edges affected by updates occupy a small proportion of the unshared partition, indicating that there is still residual data in the unshared partition that can be shared across all n jobs. Within the CGCGraph workflow, the n graph snapshots are first divided into a shared graph that overlaps among all snapshots and n distinct unshared graphs. The shared graph is then partitioned to fit the GPU memory capacity and sequentially transferred to the GPU for computation. At the same time, n unshared graphs reside on the CPU and are processed by n threads. With CGCGraph, the transfer overhead of unshared data is decreased.

Insufficient utilization of computational resources. While existing graph snapshot processing systems implement concurrent graph processing, they do not fully utilize the computational resources. In the current execution mode, the GPU handles shared graph data concurrently, but

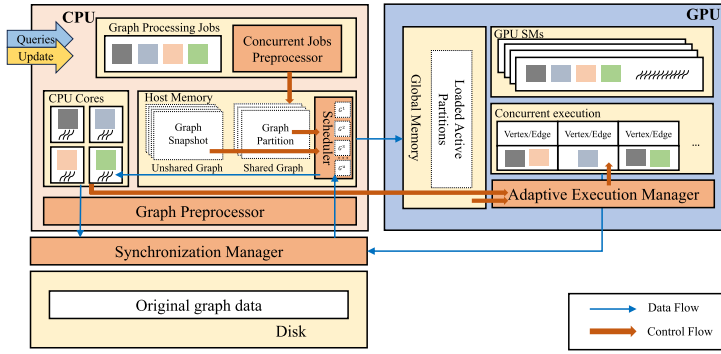


Fig. 3. Overview of CGCGraph.

processes unshared data serially, leading to inefficiencies. Under the circumstances, in the stage of processing shared graph data, the GPU concurrently processes n snapshots, effectively leveraging computational resources. However, in the stage of processing unshared graph data, the sequential computation of snapshots leads to inefficient resource utilization. Figure 2(c) shows the fluctuations in GPU utilization during the parallel SSSP job execution on the twitter-2010 dataset, where the GPU utilization demonstrates regular periodic fluctuations during execution. Each period corresponds to one iteration of the graph processing. When computing the shared graph concurrently, the GPU utilization approaches full capacity. In contrast, serial computation of the unshared graph results in the GPU utilization of less than 50%. The issues mentioned above suggest that the transferring the unshared graphs to the GPU may not be appropriate for concurrent processing of dynamic graph snapshots.

In addition, Figure 2(b) shows the utilization timeline of different CPU cores in one iteration during the concurrent execution of SSSP jobs on twitter-2010 dataset. The horizontal axis represents time and the vertical axis represents different CPU cores. In Figure 2(b), the black part indicates that the CPU core is performing computation, while the white part indicates that the CPU core is idle. It is obviously observed that most CPU cores are idle during the concurrent execution of SSSP jobs. Given that the unshared graphs require less computational resource than shared graph, it is feasible to offload the processing of the unshared graph to the CPU.

3 CGCGraph

CGCGraph is a *runtime software system* that can be seamlessly compatible with existing GPU-based graph processing frameworks. In this section, we introduce the architecture and main components of our CGCGraph system.

3.1 System Architecture

Figure 3 illustrates the high-level design of CGCGraph. It consists of four main components: Pre-processor (Current Jobs Preprocessor and Graph Preprocessor), Scheduler, Synchronization Manager and Adaptive Execution Manager. The *Graph Preprocessor* and *Concurrent Jobs preprocessor* are used to divide the graph data according to the batched jobs, and the *Scheduler* sends the graph data to either the CPU or the GPU. The *Adaptive Execution Manager* supports concurrent graph processing on SMs. In addition, several specific optimizations are provided by *Adaptive Execution Manager* in Section 4, making the execution of CGCGraph more efficient. The *Synchronization Manager* is used to synchronize the data between the CPU and the GPU.

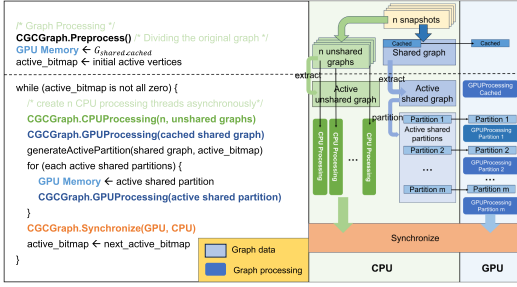


Fig. 4. Workflow of CGCGraph, showing how CGC-Graph can concurrently process multiple dynamic graph processing jobs.

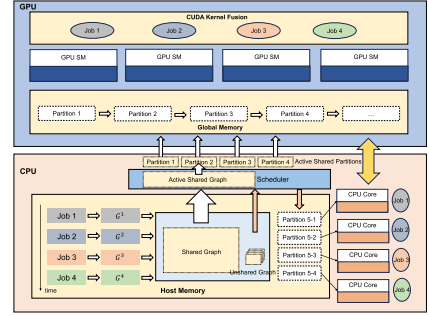


Fig. 5. An example of CGCGraph scheduling graph snapshots.

To better elucidate the execution process of CGCGraph, Figure 4 illustrates the workflow of concurrent dynamic graph processing. CGCGraph adopts an “iterative-convergent” processing schema akin to the one delineated in Section 2.2. The vertices which need to be processed in the current iteration are referred to as active vertices and are stored using an active_bitmap, where the corresponding bit of the active vertex is 1, otherwise 0.

Before performing the graph algorithm, the Graph Preprocessor computes the vertices that need to be cached on the GPU side and sends their out edges to the GPU, which will be described in Section 3.2. At the beginning of each iteration, the Scheduler extracts the vertices with corresponding value 1 in active_bitmap and their adjacent edges from the **shared graph**, which is formed as **active shared graph**. Likewise, the **active unshared graphs** are extracted. Then, the active unshared graphs reside on the CPU side and the API *CPUProcessing()* is called to calculate them. Meanwhile, the active shared graph is sent to the GPU side. The API *GPUProcessing()* is called to process the graph data cached by the Graph Preprocessor and the active shared graph. In the process, the active vertices of the next iteration are generated and stored using next_active_bitmap. Since the size of the active shared graph may exceed the GPU memory size, we further partition it as multiple active shared partitions. Specifically, for n snapshots, *CPUProcessing()* creates n CPU threads to perform the Expand operation. *GPUProcessing()* utilizes the kernel fusion technique to concurrently execute the Expand operation of n snapshots in a CUDA Kernel. The Adaptive Execution Manager maintains the life cycle of CPU/GPU processing(), handling active vertices and generating new ones for the subsequent iteration. Once n threads created by *CPUProcessing()* finish their work and *GPUProcessing()* has concurrently processed all active shared graph partitions, the Synchronization Manager synchronizes between the CPU and GPU. After synchronization is complete, the next iteration will be performed. Convergence is reached when either the state data of all vertices do not change (the number of active vertices in the next iteration is 0) or when the procedure reaches a limited iteration time.

To integrate CGCGraph with an existing graph processing system, we design five APIs and apply kernel fusion to modify the graph algorithm kernel. Figure 6 illustrates the integration of CGCGraph into Subway [29].

3.2 Graph Preprocessing

CGCGraph assumes that graph data can be placed entirely in host memory and not entirely in GPU global memory.

In the preprocessing phase, Concurrent Jobs Preprocessor in Figure 3 packs graph processing jobs in the job queue into a batch and arranges them in ascending order according to the snapshot

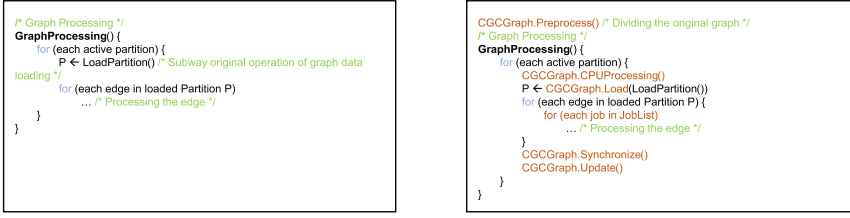


Fig. 6. An example to show how CGCGraph can be integrated into Subway.

timestamp. Based on the insertions and deletions among snapshots, Graph Preprocessor divides the original snapshots into the shared data and the unshared data. Before CGCGraph's workflow, Concurrent Jobs Preprocessor packs n graph processing jobs into a batch. In the workflow shown in Figure 4, *CGCGraph.Preprocess()* performs the functions of the Graph Preprocessor. For graph snapshots with batchsize n , *CGCGraph.Preprocess()* divides them into one shared graph and n unshared graphs.

For correctness in dynamic graph processing, vertex pairs are created to synchronize new state arrays of shared graph data and unshared graph data. Considering two edges, denoted as e_1 in the shared graph data and e_2 in the unshared graph data, respectively. If the destination vertex v_d of e_1 and the source vertex v_s of e_2 are the same, a vertex pair is established between v_d and v_s . Similarly, for the edge e_3 in the unshared graph data and e_4 in the shared graph data, if the destination vertex $v_{d'}$ of e_3 and the source vertex $v_{s'}$ of e_4 are the same, a vertex pair is established between $v_{d'}$ and $v_{s'}$. In these cases of vertex pairs, v_d and $v_{d'}$ are the master vertices, while v_s and $v_{s'}$ are the mirror vertices. When the synchronization operation is performed at the end of each iteration, the data communication occurs only between the vertices in vertex pairs, and the master vertex sends the new state data to the mirror vertex.

Since real-world graphs follow the power-law degree distribution, high-degree vertices are visited frequently [42]. In concurrent graph processing scenarios, these high-degree vertices are visited more frequently due to multiple jobs processing the graph concurrently. Consequently, performance gains can be achieved by allocating a portion of the GPU's global memory for caching these high-degree vertices, which can be shared by concurrent jobs. The size of the GPU global memory used for caching is user-specified. Hereafter, *Cached* denotes the proportion of global memory used for caching, $|Global|$ denotes the size of the GPU global memory. Specifically, vertices are first sorted in descending order of in-degree. Based on the in-degree descending reordering arrays, we accumulate the out-degrees of vertices until the sum of the out-degree reaches $\frac{Cached \times |Global|}{M}$, where M is the memory size occupied by one edge. The number of vertices involved in the accumulation is recorded as V_{cached} and all outgoing edges of the first $V_{cached} - 1$ vertices are cached. After calculating the cached data, it will be sent to the GPU memory in the preprocessing phase. The cached data is part of the shared graph and is represented using a dark blue box in Figure 4. After the preprocessing phase, the initial query vertices of each job are used to initialize the active vertices of the first iteration.

3.3 Scheduler

The scheduling model of CGCGraph is shown in Figure 5, users submitted four query jobs for snapshots G^1 - G^4 , and in the graph preprocessing phase, the four graph snapshots were partitioned into shared and unshared graphs. Shared graphs and unshared graphs are stored in host memory. In each iteration of the "iteration-convergence" process, the Scheduler schedules the graph data to the GPU or CPU side.

For the shared graph, since only a portion of the vertices are active during each iteration of the graph algorithm, it would waste a large amount of CPU-GPU transfer bandwidth if all of them are transferred to the GPU. Therefore, CGCGraph dynamically partitions the shared graph data in each iteration according to the distribution of active vertices. The scheduler extracts the **active shared graph** from the **shared graph** according to the value of `active_bitmap`, as illustrated above. This process corresponds to the Filter process in Filter-expand described in Section 2.2. In order to transfer the active shared graph to the GPU, the Scheduler needs to determine the partition size by considering the available global memory capacity of the GPU. Assuming that the reserved space on the GPU is r and the size used to store the vertex state data is s , the size of the available global memory is $(1-Cached) \times |Global| - r - s$. Based on the `active_bitmap`, CGC-Graph uses the active vertices to compute the active edges that need to be processed in the current iteration. At the same time, **active shared partitions** are constructed based on the available global memory size. The number of edges in each active shared partition is $\frac{(1-Cached) \times |Global| - r - s}{M}$, where M is the memory size occupied by one edge. In this way, for each shared partition, only one copy of the graph structure data needs to be transferred to the GPU, and the graph data is accessed by multiple concurrent jobs. For the example in Figure 5, the active shared graph is divided into four active shared partitions (Partitions 1, 2, 3, and 4), which are then sent to the GPU's global memory in sequence. Subsequently, job1 - job4 concurrently compute the active shared partitions in the manner of kernel fusion, and they complete the computations using the SMs on the GPU.

For unshared graph data, the Scheduler constructs the **active unshared graphs** based on the active vertex bitmap for each job in the current iteration. Then, the unshared active graph data corresponding to each snapshot is processed by a CPU thread. Although the unshared graph does not have to be transferred to the GPU, extracting the active subgraphs still reduces redundant computations on each CPU thread. In the host memory, the scheduler constructs the active unshared graph partitions 5-1 to 5-4 from the four unshared graphs respectively based on the active vertex bitmap, and they will be assigned to 4 independent threads for parallel computation.

3.4 Adaptive Execution

After preprocessing and scheduling, the graph data to be processed in a graph processing iteration includes the *active unshared graph* residing on the CPU side, the *cached graph* stored on the GPU side, and the *active shared graph* that needs to be partitioned and transferred to the GPU side.

In each iteration of the graph processing, the Adaptive Execution Manager processes three kinds of the graph data. To fully utilize the computational resources, the processing on the CPU and GPU is performed simultaneously.

Processing on CPU. The Adaptive Execution Manager uses *CGCGraph.CPUProcessing(n , unshared graphs)* in Figure 4 to process the active unshared graph on the CPU. *CGC-Graph.CPUProcessing(n , unshared graphs)* creates n CPU threads for n unshared active graphs and each thread executes the corresponding EXPAND operation.

Processing on GPU. While the CPU computes the unshared graph, the GPU side needs to process the active shared graph and cached graph. The Adaptive Execution Manager invokes *CGC-Graph.GPUProcessing(cached shared graph)* in Figure 4 to handle the cached graph, then calls *CGC-Graph.GPUProcessing(active shared partition)* for each active shared partition generated by the Scheduler. The cached graph can be considered as a separate graph partition.

In traditional graph processing systems, each job has an independent CUDA Kernel, which operates Filter-Expand separately. Since each job has a separate memory space, the same graph partition is repeatedly loaded into the GPU by different jobs. This simple method of executing concurrent jobs independently results in redundant data transfer.

ALGORITHM 2: Execution model for a single round iteration using reordered state arrays and assemble existence bitmaps

Input: $G_{shared_partition} = (V, E, W)$; job_lists ; $state_arrays$;
 $sharedActiveVertices$ and $existenceBitmap$;

Output: handling strategy of each vertex in $Bitmap_label_union$;

```

1:  $len = job\_lists.len$ ;
2:  $size = \lceil \frac{len \cdot 2}{8} \rceil \cdot 8$ ;
3:  $bias = 0.5 \cdot size$ ;
4: for  $v_s$  in  $sharedActiveVertices$  do
5:   for  $v_d$  in  $v_s.outEdges$  do
6:     for  $job_j$  in  $job\_lists$  do
7:       if  $existenceBitmaps_{size \cdot s + j} == 1$  then
8:          $EXPAND(v_s, v_d, state\_arrays, existenceBitmaps)$ ;
9:       end if
10:    end for
11:  end for
12: end for
13: function  $EXPAND(v_s, v_d, state\_arrays, existenceBitmaps)$ 
14:    $distance = state\_arrays[s][j] + v_d.weight$ 
15:   if  $distance < state\_arrays[d][j]$  then
16:      $atomic(state\_arrays[d][j] = distance)$ ;
17:      $atomic(existenceBitmaps_{size \cdot d + bias + j} = 1)$ ;
18:   end if
19: end function

```

To mitigate this problem, we use CUDA kernel fusion to force multiple concurrent jobs to unify data access paths. During one iteration of the graph processing, the shared graph partition is loaded into the global memory of the GPU, followed by a series of update operations that execute concurrently across multiple jobs. The specific execution algorithm is shown in Algorithm 2, and $sharedActiveVertices$ represents the union frontier of all jobs in the current iteration. To ensure that all jobs unify edge access paths, we visit each active vertex v in the $sharedActiveVertices$ in a common order, and use a union kernel function to trigger the jobs that need to process vertex v . Subsequently, the triggered jobs perform concurrently. In this way, the data of active edge loaded to the GPU can be shared by the kernels of all jobs. As a result, this execution method mitigates the redundant loading of graph partitions shared by multiple jobs. The Adaptive Execution Manager is also responsible for performing runtime adjustments, and the specific optimization methods are detailed in Sections 4.2 and 4.3.

3.5 Synchronization of CPU and GPU

After processing the shared graph on the GPU and the unshared graph on the CPU, synchronization between the CPU and GPU is needed to acquire the updated vertex states and the active vertex bitmap. For synchronization of vertex states, this synchronization strategy focuses solely on the vertex pairs identified during the preprocessing phase, thereby reducing the overhead compared to synchronizing all vertices. To further minimize the synchronization overhead, the new active vertex bitmap is used to filter out the vertices that have not updated their state in the current iteration, so that only vertices pairs whose state has been updated need to communicate.

Specifically, the CPU-GPU synchronization overhead includes three parts: **transfer of state data between vertex pairs**, **transfer of active vertex bitmaps**, and **update of active vertex bitmaps and state arrays**. Among them, the transfer of state data and active vertex bitmap between CPU and GPU is finished by calling `cudaMemcpy()`. In particular, the state data is packaged and sent after all jobs have been executed, allowing batch transmission and reducing transfer overhead. The update of active vertex bitmaps on the CPU side is completed using the bitmap update *Bitmap_label_union* for each job. Using vertex pairs, unnecessary data transfers are reduced, decreasing the synchronization overhead.

3.6 Profiling Phase

The Profiling phase is a pre-analysis phase conducted prior to the actual operation of the CGC-Graph system. In the profiling phase, analysis is performed to select specific parameters for CGC-Graph runtime to obtain optimal performance for the actual execution. The parameters needed to be identified comprises the proportion of global memory for caching (in Section 3.2), the threshold h_0 for imbalance (in Section 4.2), the threshold d_0 for average out-degree (in Section 4.2), and the parameters within the prediction model (in Section 4.3). Currently, the profiling phase specifically conducts graph processing with varying parameter characteristics on five datasets, including LiveJournal [41], in-2004 [5], eu-2005 [2], uk-2005 [4], and uk-2006 [4].

Because the cache data proportion, imbalance index, and average degree are all acted upon at the CUDA kernel level, their values are less closely related to the algorithm and dataset type and can be obtained directly during the profiling phase. For example, we set different cache proportions ranging from 5% to 50% for caching data and run concurrent graph queries under different cache proportions to select the optimal cache proportion. The thresholds for the imbalance index and average degree are also generated in the same way. The acquisition of predictive model parameters is achieved through an additional training step. When we run concurrent graph queries on these datasets and algorithms, we collect specific parameters and execution times for each kernel and use them to train a simple linear regression model. During the actual operation of CGGGraph, when a user submits a graph query, CGGGraph can directly utilize the thresholds and model parameters acquired in the profiling phase.

4 Optimizing Techniques

Concurrent CPU-GPU co-execution for dynamic graph processing still faces challenges in storage, computation, and synchronization. For storage, different jobs access the same state data and active vertex bitmap with poor locality. Besides, the simple vertex bitmap will generate a large number of access conflicts generated by atomic operations during the execution of the CUDA kernel, which will affect the performance. In terms of computation, the power-law distribution of the graph leads to an imbalanced workload among threads on the GPU, which affects the execution efficiency of concurrent jobs. In addition, CPU and GPU execution patterns can be improved to increase resource utilization. As for synchronization, we have shown that in most cases the synchronization overhead is less than the overhead of sequentially transferring the unshared graph. However, in some iterations of graph processing, there may be cases where the synchronization overhead is too high. To address the above challenges, we propose three optimizing techniques to enhance the efficiency of concurrent graph processing and improve the resource utilization.

4.1 Storage: Assemble Existence Bitmap and Reordered State Array

In CGCGraph, we use existence bitmaps to record the active vertices of each job and their union, such as *active_bitmap* previously mentioned. Also, each job must have its vertex state array to record state data (e.g., the shortest distance in the SSSP algorithm). Traditionally, active vertex

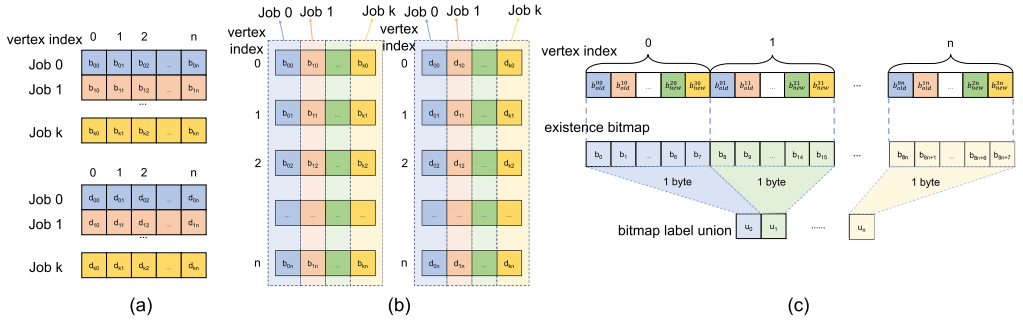


Fig. 7. (a) Existence bitmap and state array. (b) Reordered existence bitmap and state array. (c) Assemble existence bitmap.

existence bitmaps and state arrays are scattered throughout storage, as shown in Figure 7(a), compromising the locality of concurrent jobs. The way of using multiple bitmaps and arrays to store active vertices and vertex state data introduces out-of-order access. Therefore, we use the reordered existence bitmap and state array as shown in Figure 7(b). The reordered existence bitmap in each row represents the active state of a vertex for multiple graph processing jobs. Utilizing reordered existence bitmap enables sequential access to multiple existence bits of consecutive jobs within the fused kernel, thereby enhancing data locality. The benefits of the reordered state array are similar. Existing work uses reordered state arrays on host memory [8].

As described in Section 3.1, `next_active_bitmap` stores the active vertices for the next iteration, sharing the same structure as `active_bitmap` in Figure 7(b). During graph processing, multiple jobs may update the same vertex's active state, represented by consecutive bits in a row of Figure 7(b). However, the minimum granularity for CUDA to write a bit is one byte and the atomic operation is needed to preserve correctness [37]. Meanwhile, when different CUDA threads modify the bits of different vertices in one byte, the atomic operation conflicts will also occur.

Therefore, the reordered existence bitmap is further optimized as the assemble existence bitmap. Since GPU threads must use atomic operations to write concurrently to adjacent bits in order to maintain correctness, one byte allocated by the assemble existence bitmap can store at most one vertex's active state, avoiding atomic conflicts in writing adjacent vertices. Meanwhile, for an active vertex, the multiple jobs are executed by kernel fusion mechanism following the same data access paths, so there is no atomic conflict among jobs when writing the same destination vertex adjacent to the same edge. In the assemble existence bitmap, for each vertex, the left half bits of bytes are used to store the bitmap of the current iteration, while the right half bits are used to store the new bitmap of the next iteration. So the assemble existence bitmap allows one byte to represent up to 4 jobs. With no more than 4 jobs, the active state of each vertex for all jobs is stored in a single byte, as shown in Figure 7(c). For more jobs, the vertex is allocated additional bytes depending on the number of jobs, and all bytes of the vertex are similarly expanded by using the left half of the bits to store the active state in the current iteration, and using the right half of the bits to store the active state in the next iteration. Naturally, $\lceil \frac{n-2}{8} \rceil \cdot 8$ bits are allocated to store the active state of one vertex for n jobs. Algorithm 2 presents the pseudo-code for the SSSP algorithm using the assemble existence bitmap and recorded state array. For the bytes corresponding to one vertex, OR operation is performed for continuous bits to derive the *Bitmap_label union*, indicating whether one vertex is active for multiple jobs. The *sharedActiveVertices* stores all active vertices obtained from *Bitmap_label union* and is shared by all jobs. For each active vertex in *sharedActiveVertices*, adjacent active edges are processed. For the destination vertex v_{d1} of an active edge, all jobs call the *EXPAND* operation following kernel fusion model (lines 4–12), where the corresponding bit

of v_{d1} will be written as 1 if one job updated the state of v_{d1} (lines 15–17). In this way, the write operations on v_{d1} by multiple jobs are sequential, eliminating atomic operations.

4.2 Computation: Load-balancing Among Threads

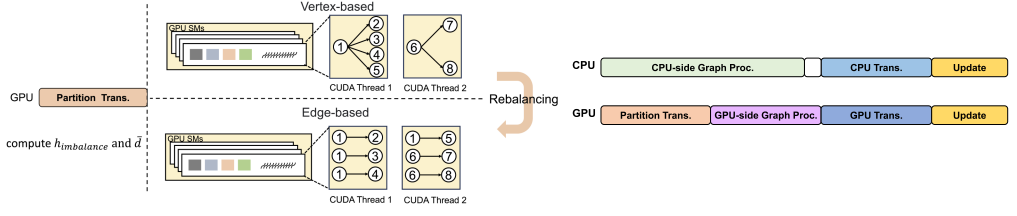
Although existing GPU-based graph processing systems have proposed a series of load-balancing strategies, they have not focused on concurrent graph processing scenarios based on kernel fusion [23, 25, 44–46]. When the Scheduler dynamically divides the active shared graph partition based on the global memory space available, the power-law distribution of the graph results in an imbalanced distribution of active edges among different partitions.

CGCGraph uses a vertex-based parallel mode for each graph partition, and for one active vertex, a CUDA thread triggers associated jobs to handle all adjacent edges. In this way, the underutilized threads remain idle until the overloaded threads finish their work. And then inter-thread synchronization is performed. Moreover, because of the kernel fusion execution, the overloaded threads are further overwhelmed, resulting in severe performance degradation.

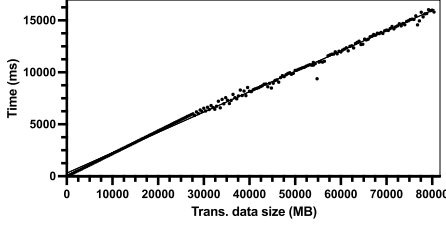
To solve this problem, CGCGraph employs a load-balancing strategy to balance the amount of computation among multiple threads. And *imbalance index* $h_{imbalance}$ is used to measure the degree of imbalance for graph partition. Specifically, for a partition P dynamically partitioned by the Scheduler, the out-degree of n vertices in P is $\{d_1, d_2, \dots, d_n\}$, respectively. The sum of the out-degrees of these n vertices is $d = \sum_{i=1}^n d_i$. The out-degree proportion of the n vertices is $\{p_1, p_2, \dots, p_n\}$, respectively, where $p_i = \frac{d_i}{d}$. Then the *imbalance index* of the partition P can be defined as $h_{imbalance} = -\sum_{i=1}^n p_i \ln(p_i)$. Firstly, if $h_{imbalance}$ is less than the threshold h_0 , the load is quite imbalanced among the vertices in partition P . Secondly, when $h_{imbalance}$ is larger than h_0 and the average out-degree $\bar{d} = \frac{d}{n}$ of the partition P is also larger than the threshold d_0 , the load will also require to be rebalanced. Specifically, the imbalance index is the entropy of all the out-degrees of the vertices in a graph partition, borrowing from the definition of entropy in informatics. Since the vertex-based parallel mode allows each thread to handle all the out-edges of a vertex when partitioning threads, the entropy of the vertex out-degrees obtained by using the degree distribution can indicate the imbalance degree among CUDA threads. The average degree serves as a supplement to the imbalance index, used to reduce the number of edges one overloaded CUDA thread needs to process with the load-balancing mechanism.

During execution, the Adaptive Execution Manager uses GPU to rapidly compute $h_{imbalance}$ and $\bar{d}$, and switches from vertex-based parallel mode to edge-based parallel mode when it needs to trigger the load-balancing mechanism. Under edge-based parallel mode, each thread processes a fixed number of edges instead of all adjacent edges of a single vertex. The partition which packed by Scheduler is further divided into several edge chunks, and the number of edges in each edge chunk is the same. For a partition where the load-balancing mechanism is triggered, all CUDA threads process an edge chunk, avoiding the skewness of computational load among threads. Compared with the vertex-based parallel mode, where threads processing vertices with small out-degree are underloaded and threads processing vertices with large out-degree are overloaded, the edge-based parallel mode balances the workload among threads. In this way, the load-balancing is improved for threads and the efficiency of concurrent graph processing jobs increases.

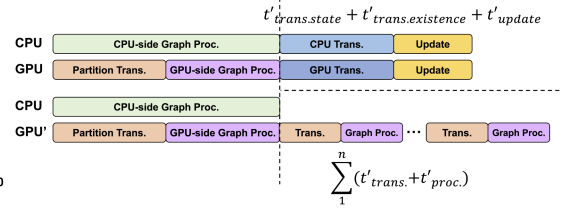
Figure 8(a) shows the process of load balancing among threads. In the vertex-based parallel mode, thread 1 processes four out-edges of vertex 1, while thread 2 handles two out-edges of vertex 6, resulting in an imbalanced workload between CUDA threads. When the load-balancing strategy is applied, both threads handle an equal number of edges. Thread 1 handles three out-edges from vertex 1, while thread 2 handles one out-edge from vertex 1 and two out-edges from vertex 6. When a CUDA Kernel switches from vertex-based parallel mode to edge-based parallel mode, additional



(a) Dynamically switch between vertex-based and edge-based parallel (b) Computational overlap between CPU and GPU in one iteration of graph processing.



(c) Relationship between cudaMemcpy() data transfer size and execution time.



(d) Prediction-based adaptive switching.

Fig. 8. Load-balancing and adaptive switching strategy.

source vertex information needs to be sent to the GPU via cudaMemcpy(), which brings additional transfer overhead. In the implementation of CGCGraph, the source vertex information sent to the GPU includes two arrays: one is the source vertex array, and the other is the offset array. For example, in thread 2 of Figure 8(a), the source vertex array is 1, 6, and the offset array is 0, 1. The transfer overhead can be reduced by compressing the source vertex information, but there is still a portion of non-negligible overhead compared to the vertex-based mode. Therefore, CGCGraph only implements the switching strategy when the degree of load imbalance is relatively high, instead of taking the edge-based parallel mode as the default one.

4.3 Synchronization: Adaptive Switching Strategy

To optimize CPU and GPU resource utilization, CGCGraph efficiently schedules execution phases across both processors. The computational overlap between CPU and GPU in one iteration of graph processing is shown in Figure 8(b). This execution mode takes full advantage of CPU and GPU computational resources to increase the efficiency of concurrent graph processing jobs. However, in certain iterations of graph processing, the additional overhead of CPU-GPU co-execution results in poorer performance than using the GPU alone, i.e., the GPU-only mode. Using GPU-only mode, in each iteration, the unshared graph on the CPU needs to be transferred to and processed by the GPU.

As mentioned earlier, the synchronization overhead introduced by CPU-GPU co-execution consists of three main components: transfer of state data between vertex pairs, transfer of active vertex bitmaps, and update of active vertex bitmaps and state arrays. To make the execution more efficient, we propose the adaptive switching strategy. By comparing the transferring time in CPU-GPU co-execution mode and GPU-only execution mode, we can predict if there is a requirement to switch execution modes. We use t' to denote the time of the prediction.

For the CPU-GPU co-execution computing mode, the synchronization overhead is

$$t'_{cpu-gpu} = t'_{trans.state} + t'_{trans.existence} + t'_{update}. \quad (1)$$

The overhead of transferring and processing the unshared graph in GPU-only mode is

$$t'_{gpu} = \sum_{i=1}^n (t'_{trans.i} + t'_{proc.i}). \quad (2)$$

Among them, $t'_{trans.state}$ and $t'_{trans.existence}$ represent the predicted transfer times for the state array and existence bitmap, respectively. t'_{update} denotes the predicted update time for each state array. $t'_{trans.i}$ and $t'_{proc.i}$ correspond to the predicted transfer and processing times for the i th unshared graph data on the GPU.

Once CPU-GPU computation completes, the number of vertex pairs to be communicated in the current iteration can be determined. If only the GPU processes the next iteration, the size of the unshared graph to be transferred can be obtained from `next_active_bitmap`. In practice, data transfer is implemented using the `cudaMemcpy()` API. To predict the consumed time of data transfer, we perform experiments on an NVIDIA A800. Specifically, data with size ranging from 4 MB to 80 GB is transferred to the GPU using `cudaMemcpy()`, and the consumed transfer time is recorded, as shown in Figure 8(c). The result shows that data transfer time and data size are linearly related, so the overhead of transfer time can be simply predicted based on data size. Presently, the number of vertex pairs needs to communicate and the number of unshared graph edges are known. Meanwhile, $t'_{trans.state}$, $t'_{trans.existence}$, and $t'_{trans.i}$ can be predicted in Equations (1) and (2).

As shown in Figure 8(b), the GPU-side update and the CPU-side update overlap in time, so the update time is $\max(t'_{update}^{cpu}, t'_{update}^{gpu})$. Since the update on the GPU is implemented in parallel via CUDA, the execution of update is faster, while the update on the CPU is slower. So the update time t'_{update} is the update time on CPU-side t'_{update}^{cpu} , which is linearly related to the number of vertex pairs. In this way, t'_{update} can be predicted. And linear parameters can be obtained during the profiling phase. Nevertheless, for the GPU-only mode, predicting graph processing time is complex and depends on the specific algorithm and kernel parameters, in addition to the number of active vertices. To predict processing time, we employ a simple linear regression model to predict the execution time of the graph processing kernel. The parameters of the linear regression model comprise the number of blocks in this CUDA kernel (nb), the number of threads (nt), the imbalance index of this partition (h), the degree of this vertex (d), the number of active edges (ne), and the type of algorithm ($algo$). The algorithm type is assigned a value of 1 for BFS, 2 for SSSP, 3 for CC, 4 for PageRank, and 5 for SSSP. Hence, the expression of the linear regression model is

$$t'_{proc.i} = w_0 \cdot nb + w_1 \cdot nt + w_2 \cdot h + w_3 \cdot d + w_4 \cdot ne + w_5 \cdot algo + w_6. \quad (3)$$

And these parameters can be obtained in advance during the profiling phase. In this way, $t'_{proc.i}$ can be predicted.

Prediction-based adaptive switching is shown in Figure 8(d). In each iteration of graph processing, when the computation on the CPU and GPU side is completed, the overhead $t'_{cpu-gpu}$ in the CPU-GPU co-execution mode and t'_{gpu} in the GPU-only mode are both predicted. If the overhead t'_{gpu} of transferring and processing the unshared graph is significantly lower than that of synchronization overhead $t'_{cpu-gpu}$, CGCGraph's computation mode will switch to GPU-only mode, and the unshared data will be transferred to the GPU for recomputation in the subsequent graph processing. Using adaptive switching strategy, execution efficiency is further improved.

5 Evaluation

In our evaluation, the host has two Gold 5218R Intel Xeon 2.1 GHz 40-core CPUs with 768 GB main memory and 10 TB secondary storage. The GPU is an NVIDIA A800 with 80 GB of global memory,

Table 1. Datasets Properties

Graph	Abbr.	Vertices	Edges	Sizes
twitter-2010 [15]	TW	41.7 M	1.5 B	24.4 GB
sk-2005 [3]	SK	50.6 M	1.9 B	32.0 GB
uk-2007 [4]	UK	105.9 M	3.7 B	62.3 GB
uk-union [4]	UU	133.6 M	5.5 B	94.6 GB
RMAT [6]	RM	134.2 M	10.7 B	183.5 GB

which is connected to the host via PCI-E 3.0. GCC 7.3.1 and CUDA 12.2 with the -O3 flag are used to compile the CPU and GPU programs.

The graphs used in our evaluation are listed in Table 1, including four real-world graphs and one synthesized graph, i.e., twitter-2010 [15], sk-2005 [3], uk-2007 [4], uk-union [4], and RMAT [6]. Furthermore, Table 1 shows the number of vertices and edges of these graph data and the size of the graph data stored with edgelist. We evaluate five common graph processing algorithms, including **breadth-first search (BFS)**, **connected components (CC)**, **single-source shortest path (SSSP)**, **single-source widest path (SSWP)**, and **PageRank (PR)**. Note that SSSP and SSWP work on weighted graphs. Therefore, the size of their input graphs is nearly double that of other algorithms.

Generating dynamic graph snapshots requires determining the size of the initial snapshot and the size of edges to add and remove in each time window. For the sake of comparison, we use a similar approach to EGraph [46] to divide the dynamic graph snapshots. The first 80% of the edges of each graph are used as the initial graph and the remaining 20% of the edges are randomly labeled with timestamps to support batch processing. In most cases, the change rate between consecutive snapshots is 1%, which consists of randomly deleting edges from 80% of the initial graph edge set and randomly inserting edges using the remaining 20% of the edge set. For twitter-2010, sk-2005, uk-2007, uk-union, and RMAT, the batch size of the snapshots is set to 8, 8, 8, 8, and 4, respectively. To evaluate performance, we submit graph processing jobs sequentially or concurrently. Since existing work typically runs the same graph algorithms, we submit the same types of graph processing jobs.

To evaluate the advantages of CGCGraph, we compare CGCGraph with three execution schemes of Subway, called *Subway-S*, *Subway-C*, and *Subway-G*. Subway-S processes graph snapshots sequentially. Subway-C processes graph snapshots concurrently but each job runs independently and does not share the shared partition data of the graph snapshots in a kernel fusion manner. And Subway-G uses an EGraph-like execution methodology for concurrent dynamic graph processing on the GPU, where jobs processing shared graph concurrently, and the unshared graphs are serially transferred to GPU for processing. In Subway-C, concurrent jobs are managed by the OS. In order to fairly illustrate the effect of CPU-GPU co-execution, the performance of Subway-S and Subway-C is adjusted to be the best. Meanwhile, the optimization strategies in Sections 4.1 and 4.2 are applied in Subway-G.

5.1 Comparison of the Overall Performance

To evaluate the effect of CGCGraph's execution mode and optimizing techniques. We conduct experiments to evaluate the overall performance. As shown in Figure 9, we evaluate the execution time of each batch of graph algorithms for multiple graph datasets. The performance of CGCGraph is superior to the other three versions, as evidenced by our evaluations. For example, compared to Subway-S, CGCGraph runs the BFS algorithm with speedup ranging from 1.8 times to 2.8 times. For the SSSP algorithm, the speedup is from 2.9 times to 4.5 times. The reason is that serially running

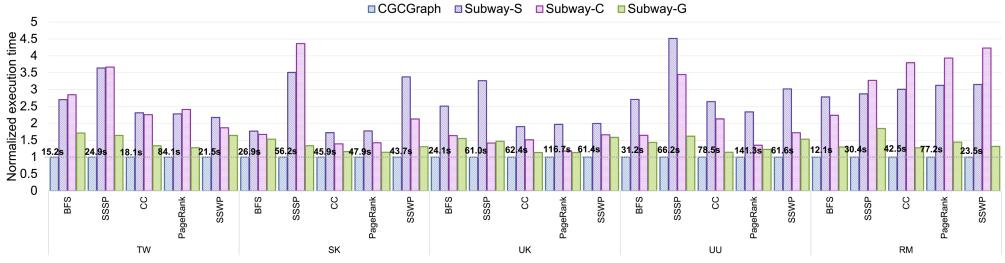


Fig. 9. Execution time of graph algorithms normalized to that of CGCGraph.

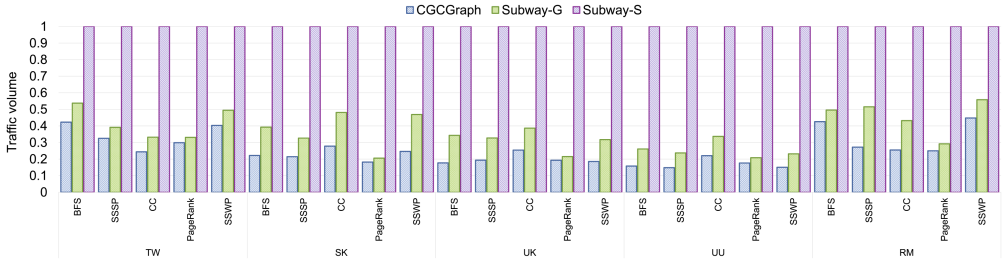


Fig. 10. Traffic volume normalized to that of Subway-S.

graph algorithms in a batch processes only one snapshot at a time, which does not fully utilize the computational resources of the NVIDIA A800 and results in lower execution performance due to underutilization.

Compared with Subway-C, CGCGraph implements concurrent BFS algorithms with speedup of 1.6 times to 2.8 times. For the SSSP algorithm, the speedup is 1.4 times to 3.7 times. Concurrent processing of graph snapshots using Subway improves the utilization of GPU computational resources and reduces the execution time of batch jobs. However, it is worth noting that Subway-C has reduced execution performance compared to Subway-S when running the SSSP algorithm on the twitter-2010, sk-2005, and RMAT datasets. This is because concurrent graph processing jobs are scheduled by the operating system and the same graph data is repeatedly loaded to the GPU, which increases the data transfer time and reduces the utilization of GPU computational resources. For different datasets, the frequency of repeatedly loading the same data is different, causing Subway-C to perform poorly on some datasets.

Compared to Subway-G, CGCGraph has a speedup of 1.3 times to 1.7 times when running the BFS algorithm and 1.3 times to 1.9 times when running the SSSP algorithm. This is because the graph snapshot data is divided into shared and unshared data in CGCGraph, while Subway-G must transfer the unshared graph data to the GPU for serial computation. This process elevates the data transfer costs between CPUs and GPUs and fails to maximize GPU resources due to the minimal computational workload of unshared graph snapshots. For the PageRank and CC algorithms, CGCGraph has speedup of 1.1 times to 1.4 times and 1.1 times to 1.3 times, respectively. Due to the high number of active vertices in the first few iterations of the PageRank and CC algorithms, the synchronization overhead between CPU and GPU increases. At the same time, increasing the number of active vertices on the CPU side increases the computational load on the CPU, resulting in performance degradation. This is the reason why the speedup of CGCGraph for PageRank and CC algorithms is relatively low.

Figure 10 shows that the traffic volume of CGCGraph is much lower than Subway-S. Since the traffic volume of Subway-S and Subway-C is essentially equivalent, Figure 10 shows only the

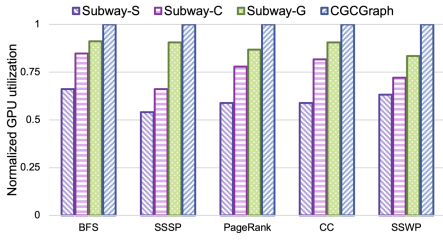


Fig. 11. Normalized GPU utilization that of CGCGraph.

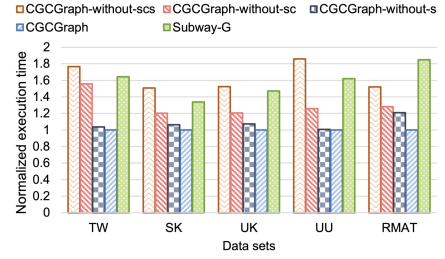


Fig. 12. Processing time of CGCGraph-without-scs, CGCGraph-without-sc, CGCGraph-without-s, CGCGraph and Subway-G normalized to that of CGCGraph for SSSP on different graphs.

data for Subway-S. Subway-G reduces traffic volume by concurrently processing shared partitions. CGCGraph further reduces traffic volume by co-executing on CPU and GPU. Taking the uk-union dataset as an example, when executing the SSSP algorithm, the traffic volume of Subway-G is 23.7% of that of Subway-S, while the traffic volume of CGCGraph is 14.8% of that of Subway-S. CGCGraph reduces traffic volume as unshared graph data no longer needs to be transferred to the GPU side for computation. Considering the average traffic volume of the five algorithms on the full dataset, Subway-G is 36.5% of Subway-S, while CGCGraph is only 25.3% of Subway-S.

We observe that CGCGraph's performance improvement is affected by the characteristics of the dataset and workload. For different algorithms running on the same dataset, when running the BFS and SSSP algorithms on the sk-2005 dataset, CGCGraph achieves 1.5 and 1.3 times faster speedups compared to Subway-G. Since CGCGraph generates more vertex pairs requiring communication when running the SSSP algorithm concurrently on sk-2005, its data transfer rate is only 65.5% of that of Subway-G, while it is 57.4% when running BFS. For the same algorithm running on different datasets, when running the SSSP algorithm on sk-2005 and RMAT, CGCGraph achieves 1.3 and 1.9 times faster speedups compared to Subway-G. The number of vertex pairs requiring communication varies across datasets when running SSSP. The data transfer rate of CGCGraph running SSSP is 65.5% of that of Subway-G on sk-2005, and 52.7% on RMAT. Since we introduce an adaptive switching strategy, the amount of data communication does not exactly correspond to the number of vertex pairs that need communication. However, it shows that CGCGraph exhibits different optimization effects due to varying communication overhead across datasets and workloads.

Figure 11 gives the average GPU utilization of the five graph algorithms on the uk-union dataset. The average utilization of Subway-S on CC jobs is only 58.8% of CGCGraph. As a comparison, the average utilization of Subway-C and Subway-G is 81.8% and 90.6% of CGCGraph, respectively. The experimental results show that CGCGraph improves the average GPU utilization of concurrent graph processing jobs.

We also evaluate the contributions of our optimizing techniques on the performance of CGCGraph. As shown in Figure 12, we sequentially remove storage, computation, and synchronization optimizations on CGCGraph. Instead of using assemble existence bitmaps, CGCGraph-without-s uses a simple bitmap; CGCGraph-without-sc uses the computation method of vertex-centric by default on this basis. And CGCGraph-without-scs further disables adaptive switching strategy. In most cases, the adaptive switching strategy and the load balancing strategy achieve better results. We note that the assemble existence bitmap does not work well in partial cases. With a batch size of 8, simply concatenating the bitmaps of the 8 jobs to exactly one byte will also effectively resolve atomic collisions.

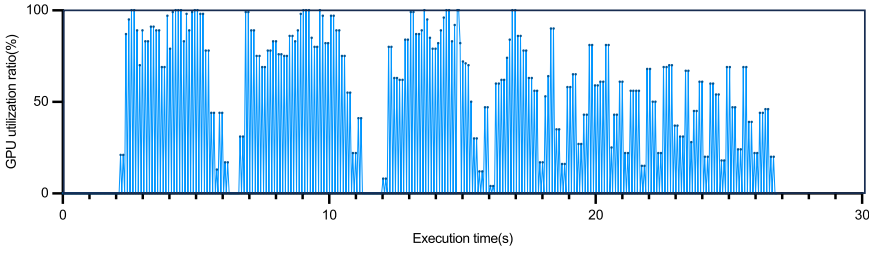


Fig. 13. GPU utilization for concurrent SSSP jobs execution (CGCGraph).

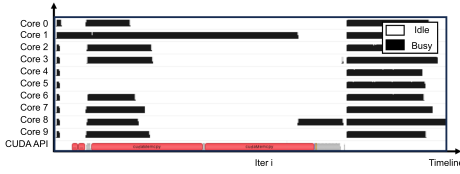


Fig. 14. CPU utilization for concurrent SSSP jobs execution (CGCGraph).

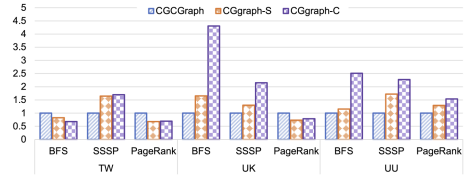


Fig. 15. Execution time of graph algorithms normalized to that of CGCGraph (Compared with CGgraph).

Case Study. To demonstrate CGCGraph's enhancement in computational resource utilization, we execute a graph processing job with a batch size of 8 on the twitter-2010 dataset. Figure 2(b) and 2(c) show the GPU and CPU utilization of Subway-G running with the same dataset and batch size. When computing the shared graph concurrently, the GPU utilization is at full capacity. By comparison, serial computing of the unshared graph leads to a GPU utilization of less than 50%. Furthermore, during the concurrent execution of SSSP jobs, most CPU cores remain idle.

After applying CGCGraph, the utilization rates of GPU and CPU are shown in Figures 13 and 14. Since the GPU only employs the method of kernel fusion to compute the shared graph, the GPU can remain at full load for the majority of one iteration of graph processing. In the execution mode of CGCGraph, the CPU is also required to participate in the computation of unshared graph data. Compared with the execution mode of Subway-G in Figure 2(b), CGCGraph can effectively enhance the utilization rate of the CPU.

5.2 Comparison with CPU-GPU Collaborative Ones

We also evaluated the current state-of-the-art CPU-GPU collaborative graph processing framework CGgraph [9]. To evaluate the advantages of CGCGraph, we compared CGCGraph with two execution schemes of CGgraph, CGgraph-S, and CGgraph-C. CGgraph-S processes graph snapshots in sequence. CGgraph-C processes graph snapshots in parallel, but each job runs independently. In CGgraph-C, concurrent jobs are managed by the operating system. As shown in Figure 15, we evaluated the execution time of each batch of graph algorithms on multiple graph datasets. In this evaluation, the batch size is set to 8.

When the scale of the graph dataset is small, the performance of CGCGraph in concurrently processing multiple graph snapshots is similar to or slightly worse than the performance of CGgraph. However, when the scale of the graph dataset increases, the performance of CGCGraph begins to surpass that of CGgraph. On the uk-union dataset, CGCGraph achieved 1.2, 1.7, and 1.3 times speedup over CGgraph-S in the BFS, SSSP, and PageRank algorithms, respectively. We noticed that the performance of CGCGraph in processing PageRank on small datasets was weaker than that of CGgraph. This is because PageRank requires more active vertex pairs for

Table 2. Preprocessing Time (Percentage of Total Execution Time)

	BFS	SSSP	CC	PageRank	SSWP
CGCGraph on twitter-2010	29.4% (21.6)	20.3% (31.2)	26.0% (24.4)	7.0% (90.4)	22.8% (27.8)
Subway-G on twitter-2010	13.6% (30.2)	9.1% (45.0)	14.5% (28.3)	4.5% (112.8)	10.4% (39.4)
CGCGraph on uk-union	26.3% (42.3)	16.8% (79.6)	16.2% (93.7)	9.7% (156.5)	19.8% (76.8)
Subway-G on uk-union	12.5% (51.1)	6.7% (115.1)	8.9% (98.7)	4.8% (182.6)	8.5% (103.1)

communication in each iteration, and the higher CPU-GPU synchronization overhead affects the performance of CGCGraph. We also noticed that due to the lack of special optimization for concurrent graph processing, the performance of CGgraph-C is generally poor.

5.3 Preprocessing Cost

The main preprocessing overheads for CGCGraph are high-degree vertices being sorted and communication vertex pairs being generated. To measure the impact of preprocessing on the overall performance, we tested the fraction of preprocessing stage in the total execution time. Note that the total execution time includes both preprocessing time and execution time.

The total execution time and preprocessing cost of CGCGraph are presented in Table 2, with the total execution time shown in parentheses (in Seconds). As observed, for the twitter-2010 dataset, the percentage of preprocessing time to total execution time for running BFS, SSSP, CC, PageRank, SSWP using CGCGraph mode is 29.4%, 20.3%, 26.0%, 7.0%, and 22.8%. For the uk-union dataset, the percentage of preprocessing time is 26.3%, 16.8%, 16.2%, 9.7%, and 19.8%. As a comparison, the preprocessing time as a percentage of the total execution time using Subway-G is 13.6%, 9.1%, 14.5%, 4.5%, and 10.4% for the twitter-2010 dataset and 12.5%, 6.7%, 8.9%, 4.8%, and 8.5% for uk-union.

5.4 Impacts of Change Rates

We evaluated CGCGraph's change rates between consecutive snapshots sensitivity by experiment with change rates ranging from 0.5% to 2%. Figure 16 presents the normalized execution time of the BFS, SSSP, CC, PageRank, and SSWP algorithms on twitter-2010 when the change rates of consecutive graph snapshots are 0.5%, 1%, and 2%. Figure 16 reveals a trend that when the change rate of graph snapshots is low, Subway-G approaches the performance of CGCGraph. However, when the change rate of snapshots increases, CGCGraph can achieve a higher speedup ratio. When the change rate is 0.5%, the speedup ratio ranges from 1.1 times to 1.4 times. When the change rate is 1%, the speedup ratio ranges from 1.3 times to 1.7 times. And when the change rate is 2%, the speedup ratio ranges from 1.2 times to 2.0 times.

When the change rate between consecutive graph snapshots is low, the adaptive switching strategy of CGCGraph will more frequently determine that the synchronization overhead between vertex pairs is higher than the overhead of transmitting unshared graph data, thus, CGCGraph will switch to GPU-only mode and approach the performance of Subway-G. However, when the change rate of snapshots increases, Subway-G incurs a higher overhead for transferring unshared graph data. At this time, the overhead of synchronous communication between vertex pairs is less than the overhead of transferring unshared graph data, and CGCGraph can achieve a better acceleration effect.

5.5 Impacts of Batch Size

We evaluated CGCGraph's batch size sensitivity by experiment with sizes ranging from 1 to 16. Figure 17 illustrates the normalized execution times for SSSP, BFS, and PageRank algorithms on

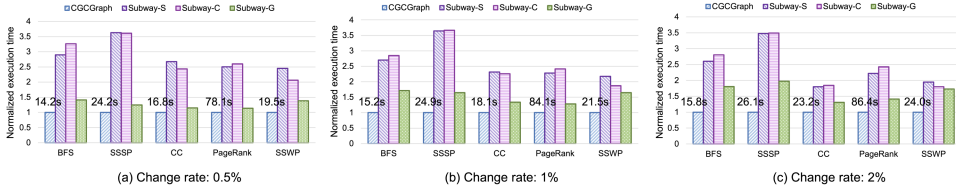


Fig. 16. (a) Execution time of graph snapshots with a change rate of 0.5% normalized to that of CGCGraph. (b) Execution time of graph snapshots with a change rate of 1% normalized to that of CGCGraph. (c) Execution time of graph snapshots with a change rate of 2% normalized to that of CGCGraph.

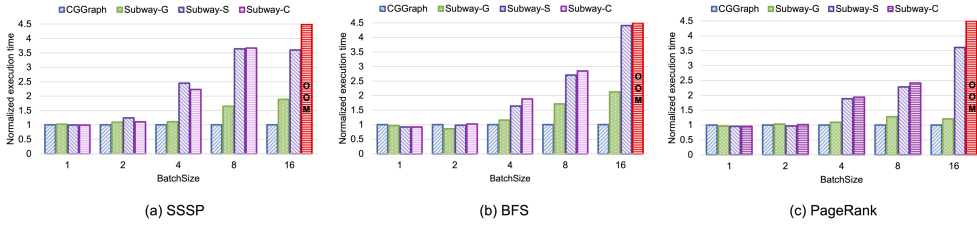


Fig. 17. Impacts of batch size on different graph processing algorithms: (a) SSSP, (b) BFS, and (c) PageRank.

twitter-2010 across these batch sizes. CGCGraph's gains are modest with small batches due to reduced redundant data transfer, where Subway-C's simple concurrency is effective. In contrast, Subway-G also yields better execution performance, which requires less unshared graph data to be transferred to the GPU for computation. As batch size grows, CGCGraph's performance enhances significantly, showing speedups of 1.9 times and 2.1 times over Subway-G for SSSP and BFS at batch size 16, compared to 1.6 times and 1.7 times at size 8. For PageRank, CPU-intensive computation limits speedup, yet a 1.2 times gain is still achieved at batch size 16.

6 Related Work

Single Graph Processing. Recently, many graph processing systems have been proposed. Ligra [30], Galois [26], and GraphMat [32] implement in-memory graph processing on the CPU. As the size of graph data grow, out-of-core graph processing systems such as GraphChi [16], X-Stream [28], and GridGraph [51] emerge. GraphChi uses sliding windows to reduce storage I/O, X-Stream eliminates random accesses through streaming, and GridGraph improves locality through chunking, further reducing IO operations. Typical graph processing frameworks on GPUs include Gunrock [39], SEP-Graph [36], LargeGraph [47] and Subway [29], and so on. Gunrock and SEP-Graph are in-memory graph processing systems that are dedicated to processing smaller-scale graph data. Subway and LargeGraph are out-of-core graph processing systems. To minimize the data transfer overhead, both of them have proposed methods for extracting the partitions of active graphs. HALO [10], Grus [37] differ from the above graph processing systems on GPUs by using a unified memory approach to data transfer. HyTGraph [38] proposes a hybrid CPU and GPU management approach for efficient data transfer to improve the performance of out-of-core graph processing on GPU platforms. To further improve the graph processing performance, TOTEM [11] implements large-scale graph processing on a hybrid CPU-GPU system. CGgraph [9] implements a CPU-GPU collaborative processing scheme. It extracts subgraphs from the full graph data and sends the subgraph to the GPU for processing, with the CPU handling the remaining parts. The vertices and edges of the subgraphs extracted by CGgraph can remain active during multiple graph iterations. CGgraph optimizes single-query performance on static graphs

but overlooks concurrent query performance on dynamic graphs. For dynamic graph processing based on graph snapshots, LLAMA [19] supports multi-version graph snapshot storage, allowing for efficient incremental snapshot updates. The incremental-based graph computation model incrementally updates the results of existing computations to achieve more efficient dynamic graph processing. Kickstarter [34] and GraphBolt [21] propose a correction-based approach that uses an efficient method to ensure the correctness of incremental processing of a graph on an evolutionary graph when the graph has deleted edges. However, these systems focus primarily on optimizing individual graph processing, leading to redundant storage consumption and data access costs when multiple concurrent graph processing jobs need to be processed.

Concurrent Graph Processing. To enable concurrent graph processing, Seraph [40] implements graph-level sharing, and decouples graph structures from graph jobs. CGraph [48] and GraphM [50] using sharing at the partition-level to fully exploit the temporal and spatial locality of shared graph access. Krill [8] adopts a vertex-level sharing approach and treats multiple graph processing jobs as a cohesive unit. This enables accessing the graph structure only once per iteration. Glign [43] uses the unified frontier to enhance the concurrent graph processing ability of Ligra, and uses the heavy iteration method to optimize the graph processing performance. GraphTune [49] introduces a distributed concurrent graph processing framework for large-scale graphs, enhancing the efficiency of distributed graph processing by prioritizing graph partitions based on dependency awareness. To process dynamic graph snapshots, EGraph [46] implements concurrent processing of dynamic graph jobs based on Subway. CommonGraph [1] processes dynamic graphs in parallel by transforming deletions into additions to find a consistent graph across snapshots.

7 Conclusion

In this article, we design CGCGraph to efficiently execute concurrent dynamic graph snapshot processing jobs. The key of CGCGraph is to realize concurrent graph processing with CPU-GPU co-execution, which can fully utilize the computational resources of CPU and GPU, reduce the data transfer, and improve the performance of graph processing jobs. The experimental results show that CGCGraph effectively improves the execution performance, reduces the data transfer volume, and enhances the computational resource utilization.

In the future, we contemplate further optimizing CGCGraph, encompassing measures such as minimizing the overhead of preprocessing and enabling support for larger-scale graph data. We can optimize the operations of vertex degree sorting and generating vertex pairs in the preprocessing phase through schemes of parallelism and CPU-GPU co-execution. For supporting larger-scale graph data, we intend to extract a smaller proxy graph from the shared graph and have the GPU solely handle the computations of this proxy graph. The complete shared graph data is distributedly stored on multiple hosts, and the correction of the proxy graph results is accomplished through distributed computing.

References

- [1] Mahbod Afarin, Chao Gao, Shafiu Rahman, Nael Abu-Ghazaleh, and Rajiv Gupta. 2023. CommonGraph: Graph analytics on evolving data. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS'23)*. Association for Computing Machinery, New York, NY, USA, 133–145. DOI: <https://doi.org/10.1145/3575693.3575713>
- [2] Paolo Boldi, Andrea Marino, Massimo Santini, and Sebastiano Vigna. 2014. BUBiNG: Massive crawling for the masses. In *Proceedings of the Companion Publication of the 23rd International Conference on World Wide Web*. International World Wide Web Conferences Steering Committee, 227–228.
- [3] Paolo Boldi, Marco Rosa, Massimo Santini, and Sebastiano Vigna. 2011. Layered label propagation: A MultiResolution coordinate-free ordering for compressing social networks. In *Proceedings of the 20th International Conference on World Wide Web*. Sadagopan Srinivasan, Krithi Ramamritham, Arun Kumar, M. P. Ravindra, Elisa Bertino, and Ravi Kumar (Eds.), ACM, 587–596.

- [4] Paolo Boldi, Massimo Santini, and Sebastiano Vigna. 2008. A large time-aware graph. *SIGIR Forum* 42, 2 (2008), 33–38.
- [5] P. Boldi and S. Vigna. 2004. The webgraph framework I: Compression techniques. In *Proceedings of the 13th International Conference on World Wide Web (WWW'04)*. Association for Computing Machinery, New York, NY, USA, 595–602. DOI: <https://doi.org/10.1145/988672.988752>
- [6] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. 2004. R-MAT: A recursive model for graph mining. In *Proceedings of the 2004 SIAM International Conference on Data Mining (SDM)*. 442–446. DOI: <https://doi.org/10.1137/1.9781611972740.43>
- [7] Anwesha Chakraborty, Trina Dutta, Sushmita Mondal, and Asoke Nath. 2018. Application of graph theory in social media. *International Journal of Computer Sciences and Engineering* 6, 10 (2018), 722–729.
- [8] Hongzheng Chen, Minghua Shen, Nong Xiao, and Yutong Lu. 2021. Krill: A compiler and runtime system for concurrent graph processing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. 1–16.
- [9] Pengjie Cui, Haotian Liu, Bo Tang, and Ye Yuan. 2024. CGgraph: An ultra-fast graph processing system on modern commodity CPU-GPU co-processor. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1405–1417. DOI: <https://doi.org/10.14778/3648160.3648179>
- [10] Prasun Gera, Hoyjong Kim, Piyush Sao, Hyesoon Kim, and David Bader. 2020. Traversing large graphs on GPUs with unified memory. *Proceedings of the VLDB Endowment* 13, 7 (2020), 1119–1133. DOI: <https://doi.org/10.14778/3384345.3384358>
- [11] Abdullah Gharaibeh, Tahsin Reza, Elizeu Santos-Neto, Lauro Beltrao Costa, Scott Sallinen, and Matei Ripeanu. 2014. Efficient Large-Scale Graph Processing on Hybrid CPU and GPU Systems. arXiv:1312.3018. Retrieved from <https://arxiv.org/abs/1312.3018>
- [12] Joseph E. Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. 2012. PowerGraph: Distributed graph-parallel computation on natural graphs. In *Proceedings of the 10th USENIX Conference on Operating Systems Design and Implementation (OSDI'12)*. USENIX Association, USA, 17–30.
- [13] Sungwon Jung and S. Pramanik. 1996. HiTi graph model of topographical road maps in navigation systems. In *Proceedings of the 12th International Conference on Data Engineering*. 76–84. DOI: <https://doi.org/10.1109/ICDE.1996.492091>
- [14] Eren Kurshan, Hongda Shen, and Haojie Yu. 2020. Financial crime and fraud detection using graph computing: Application considerations and outlook. In *Proceedings of the 2020 2nd International Conference on Transdisciplinary AI (TransAI'20)*. 125–130. DOI: <https://doi.org/10.1109/TransAI49837.2020.00029>
- [15] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a social network or a news media?. In *Proceedings of the 19th International Conference on World Wide Web (WWW'10)*. Association for Computing Machinery, New York, NY, USA, 591–600. DOI: <https://doi.org/10.1145/1772690.1772751>
- [16] Aapo Kyrola, Guy Blelloch, and Carlos Guestrin. 2012. GraphChi: Large-scale graph computation on just a PC. In *Proceedings of the 10th USENIX Conference on Operating Systems Design and Implementation (OSDI'12)*. USENIX Association, USA, 31–46.
- [17] Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. 2005. Graphs over time: Densification laws, shrinking diameters and possible explanations. In *Proceedings of the 11th ACM SIGKDD International Conference on Knowledge Discovery in Data Mining (KDD'05)*. Association for Computing Machinery, New York, NY, USA, 177–187. DOI: <https://doi.org/10.1145/1081870.1081893>
- [18] Yucheng Low, Danny Bickson, Joseph Gonzalez, Carlos Guestrin, Aapo Kyrola, and Joseph M. Hellerstein. 2012. Distributed graphlab: A framework for machine learning and data mining in the cloud. *Proceedings of the VLDB Endowment* 5, 8 (2012), 716–727. DOI: <https://doi.org/10.14778/2212351.2212354>
- [19] Peter Macko, Virendra J. Marathe, Daniel W. Margo, and Margo I. Seltzer. 2015. LLAMA: Efficient graph analytics using large multiversioned arrays. In *Proceedings of the 2015 IEEE 31st International Conference on Data Engineering*. 363–374. DOI: <https://doi.org/10.1109/ICDE.2015.7113298>
- [20] Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. 2010. Pregel: A system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data (SIGMOD'10)*. Association for Computing Machinery, New York, NY, USA, 135–146. DOI: <https://doi.org/10.1145/1807167.1807184>
- [21] Mugilan Mariappan and Keval Vora. 2019. GraphBolt: Dependency-driven synchronous processing of streaming graphs. In *Proceedings of the 14th EuroSys Conference 2019 (EuroSys'19)*. Association for Computing Machinery, New York, NY, USA, Article 25, 16 pages. DOI: <https://doi.org/10.1145/3302424.3303974>
- [22] Robert Ryan McCune, Tim Weninger, and Greg Madey. 2015. Thinking like a vertex: A survey of vertex-centric frameworks for large-scale distributed graph processing. *ACM Computing Surveys* 48, 2, Article 25 (2015), 39 pages. DOI: <https://doi.org/10.1145/2818185>
- [23] Duane Merrill, Michael Garland, and Andrew Grimshaw. 2012. Scalable GPU graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'12)*. Association for Computing Machinery, New York, NY, USA, 117–128. DOI: <https://doi.org/10.1145/2145816.2145832>

- [24] Youshan Miao, Wentao Han, Kaiwei Li, Ming Wu, Fan Yang, Lidong Zhou, Vijayan Prabhakaran, Enhong Chen, and Wenguang Chen. 2015. Immortalgraph: A system for storage and analysis of temporal graphs. *ACM Transactions on Storage (TOS)* 11, 3 (2015), 1–34.
- [25] Rupesh Nasre, Martin Burtcher, and Keshav Pingali. 2013. Data-driven versus topology-driven irregular computations on GPUs. In *Proceedings of the 2013 IEEE 27th International Symposium on Parallel and Distributed Processing*. 463–474. DOI : <https://doi.org/10.1109/IPDPS.2013.28>
- [26] Donald Nguyen, Andrew Lenharth, and Keshav Pingali. 2013. A lightweight infrastructure for graph analytics. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP'13)*. Association for Computing Machinery, New York, NY, USA, 456–471. DOI : <https://doi.org/10.1145/2517349.2522739>
- [27] Naoto Ohsaka, Takanori Maehara, and Ken-ichi Kawarabayashi. 2015. Efficient PageRank tracking in evolving networks. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'15)*. Association for Computing Machinery, New York, NY, USA, 875–884. DOI : <https://doi.org/10.1145/2783258.2783297>
- [28] Amitabha Roy, Ivo Mihailovic, and Willy Zwaenepoel. 2013. X-Stream: Edge-centric graph processing using streaming partitions. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP'13)*. Association for Computing Machinery, New York, NY, USA, 472–488. DOI : <https://doi.org/10.1145/2517349.2522740>
- [29] Amir Hossein Nodehi Sabet, Zhijia Zhao, and Rajiv Gupta. 2020. Subway: Minimizing data transfer during out-of-GPU-Memory graph processing. In *Proceedings of the 15th European Conference on Computer Systems (EuroSys'20)*. Association for Computing Machinery, New York, NY, USA, Article 12, 16 pages. DOI : <https://doi.org/10.1145/3342195.3387537>
- [30] Julian Shun and Guy E. Blelloch. 2013. Ligra: A lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 135–146.
- [31] Joakim Skarding, Bogdan Gabrys, and Katarzyna Musial. 2021. Foundations and modeling of dynamic networks using dynamic graph neural networks: A survey. *IEEE Access* 9 (2021), 79143–79168. DOI : <https://doi.org/10.1109/ACCESS.2021.3082932>
- [32] Narayanan Sundaram, Nadathur Satish, Md Mostofa Ali Patwary, Subramanya R. Dulloor, Michael J. Anderson, Satya Gautam Vadlamudi, Dipankar Das, and Pradeep Dubey. 2015. GraphMat: High performance graph analytics made productive. *Proceedings of the VLDB Endowment* 8, 11 (2015), 1214–1225. DOI : <https://doi.org/10.14778/2809974.2809983>
- [33] Keval Vora, Rajiv Gupta, and Guoqing Xu. 2016. Synergistic analysis of evolving graphs. *ACM Transactions on Architecture and Code Optimization* 13, 4 (2016), 1–27.
- [34] Keval Vora, Rajiv Gupta, and Guoqing Xu. 2017. KickStarter: Fast and accurate computations on streaming graphs via trimmed approximations. In *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'17)*. Association for Computing Machinery, New York, NY, USA, 237–251. DOI : <https://doi.org/10.1145/3037697.3037748>
- [35] Guan Wang, Sihong Xie, Bing Liu, and Philip S. Yu. 2012. Identify online store review spammers via social review graph. *ACM Transactions on Intelligent Systems and Technology* 3, 4 (2012), 1–21.
- [36] Hao Wang, Liang Geng, Rubao Lee, Kaixi Hou, Yanfeng Zhang, and Xiaodong Zhang. 2019. SEP-graph: Finding shortest execution paths for graph processing under a hybrid framework on GPU. In *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*. 38–52.
- [37] Pengyu Wang, Jing Wang, Chao Li, Jianzong Wang, Haojin Zhu, and Minyi Guo. 2021. Grus: Toward unified-memory-efficient high-performance graph processing on GPU. *ACM Transactions on Architecture and Code Optimization* 18, 2, Article 22 (2021), 25 pages. DOI : <https://doi.org/10.1145/3444844>
- [38] Qiange Wang, Xin Ai, Yanfeng Zhang, Jing Chen, and Ge Yu. 2023. HyTGraph: GPU-accelerated graph processing with hybrid transfer management. In *Proceedings of the 2023 IEEE 39th International Conference on Data Engineering (ICDE'23)*. 558–571. DOI : <https://doi.org/10.1109/ICDE55515.2023.00049>
- [39] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. 2016. Gunrock: A high-performance graph processing library on the GPU. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 1–12.
- [40] Jilong Xue, Zhi Yang, Zhi Qu, Shian Hou, and Yafei Dai. 2014. Seraph: An efficient, low-cost system for concurrent graph processing. In *Proceedings of the 23rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC'14)*. Association for Computing Machinery, New York, NY, USA, 227–238. DOI : <https://doi.org/10.1145/2600212.2600222>
- [41] Jaewon Yang and Jure Leskovec. 2012. Defining and evaluating network communities based on ground-truth. In *Proceedings of the 2012 IEEE 12th International Conference on Data Mining*. 745–754. DOI : <https://doi.org/10.1109/ICDM.2012.138>

- [42] Yuichiro Yasui and Katsuki Fujisawa. 2015. Fast and scalable NUMA-based thread parallel breadth-first search. In *Proceedings of the 2015 International Conference on High Performance Computing and Simulation (HPCS'15)*. 377–385. DOI : <https://doi.org/10.1109/HPCSim.2015.7237065>
- [43] Xizhe Yin, Zhijia Zhao, and Rajiv Gupta. 2022. Glign: Taming misaligned graph traversals in concurrent graph processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 78–92.
- [44] Jie Zhang, Yuan Zhang, Yiming Sun, Huawei Cao, Xuejun An, and Xiaochun Ye. 2022. Parallel-friendly and work-efficient single source shortest path algorithm on single-node system. In *Proceedings of the 2022 IEEE 24th International Conference on High Performance Computing and Communications; 8th International Conference on Data Science and Systems; 20th International Conference on Smart City; 8th International Conference on Dependability in Sensor, Cloud and Big Data Systems and Application (HPCC/DSS/SmartCity/DependSys'22)*. 1115–1122. DOI : <https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys57074.2022.00176>
- [45] Yuan Zhang, Huawei Cao, Jie Zhang, Yiming Sun, Ming Dun, Junying Huang, Xuejun An, and Xiaochun Ye. 2023. A bucket-aware asynchronous single-source shortest path algorithm on GPU. In *Proceedings of the 52nd International Conference on Parallel Processing Workshops (ICPP Workshops'23)*. Association for Computing Machinery, New York, NY, USA, 50–60. DOI : <https://doi.org/10.1145/3605731.3605746>
- [46] Yu Zhang, Yuxuan Liang, Jin Zhao, Fubing Mao, Lin Gu, Xiaofei Liao, Hai Jin, Haikun Liu, Song Guo, Yangqing Zeng, et al. 2022. Egraph: Efficient concurrent GPU-based dynamic graph processing. *IEEE Transactions on Knowledge and Data Engineering* 35, 6 (2022), 5823–5836.
- [47] Yu Zhang, Da Peng, Xiaofei Liao, Hai Jin, Haikun Liu, Lin Gu, and Bingsheng He. 2021. LargeGraph: An efficient dependency-aware GPU-accelerated large-scale graph processing. *ACM Transactions on Architecture and Code Optimization* 18, 4, Article 58 (2021), 24 pages. DOI : <https://doi.org/10.1145/3477603>
- [48] Yu Zhang, Jin Zhao, Xiaofei Liao, Hai Jin, Lin Gu, Haikun Liu, Bingsheng He, and Ligang He. 2019. CGraph: A distributed storage and processing system for concurrent iterative graph analysis jobs. *ACM Transactions on Storage* 15, 2 (2019), 1–26.
- [49] Jin Zhao, Yu Zhang, Ligang He, Qikun Li, Xiang Zhang, Xinyu Jiang, Hui Yu, Xiaofei Liao, Hai Jin, Lin Gu, Haikun Liu, Bingsheng He, Ji Zhang, Xianzheng Song, Lin Wang, and Jun Zhou. 2023. GraphTune: An efficient dependency-aware substrate to alleviate irregularity in concurrent graph processing. *ACM Transactions on Architecture and Code Optimization* 20, 3, Article 37 (2023), 24 pages. DOI : <https://doi.org/10.1145/3600091>
- [50] Jin Zhao, Yu Zhang, Xiaofei Liao, Ligang He, Bingsheng He, Hai Jin, Haikun Liu, and Yicheng Chen. 2019. GraphM: An efficient storage system for high throughput of concurrent graph processing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. 1–14.
- [51] Xiaowei Zhu, Wentao Han, and Wenguang Chen. 2015. {GridGraph}:{Large-Scale} graph processing on a single machine using 2-level hierarchical partitioning. In *Proceedings of the 2015 USENIX Annual Technical Conference (USENIX ATC'15)*. 375–386.

Received 31 July 2024; revised 29 November 2024; accepted 27 May 2025